第5章　多态性

第五章 多态性

1. 基本概念
2. 运算符重载
3. 虚函数
4. 抽象类

1. 基本概念

什么是多态性

- 面向对象程序设计三大基本特征：封装、继承和多态性
- 多态性：相同的消息(函数调用)，产生不同的动作(函数实现)
  - 多态性指用同一函数名定义的多个函数体，分别执行不同但类似的操作，即用同样的接口访问功能不同的函数
  - 两种多态性
    - ✓编译时多态性：编译时就确定调用哪一个同名函数【重载】
    - ✓运行时多态性：执行时才确定调用哪一个同名函数【虚函数】

```
class A{
public:
  void f(){…}
  void f(int x){…}
};
A a; a.f(); a.f(1);
```

```
class A{
 public:
 virtual void f(){…}
};
class B:public A{
public:
void f(){…}
};
A a, *pa; B b;
pa=&a; pa->f(); //执行A::f
Pa=&b; pa->f(); //执行B::f
```

1. 基本概念

静态联编与动态联编

- 源程序经过编译、连接，成为可执行文件的过程是把可执行代码联编(或称装配)在一起的过程——联编
- 两类“联编”
  - 程序运行前就完成的联编叫静态联编，又叫前期联编；静态联编支持的多态性称为编译时多态性，也称静态多态性【重载】

    优点：速度快
  - 程序运行时才完成的联编叫动态联编，又叫后期联编；动态联编支持的多态性称为运行时多态性，也称动态多态性【虚函数】

    优点：灵活

静态联编：函数重载；运算符重载【本质是函数重载】

动态联编 ⟶ 虚函数

第五章 多态性

1. 基本概念
2. 运算符重载
3. 虚函数
4. 抽象类

2. 运算符重载

运算符重载的概念

```
class C{
 float re;
 float im;
public:
 C operator+(C& c){…}
};
```

```
C c1,c2,c3;
c3=c1+c2;
```

- 运算符重载：在原运算符含义基础上，再定义对用户自定义类型进行操作
- 运算符重载，优先级、结合性、操作数个数不变
- 大多数运算符可重载，但少数运算符不能重载：
  - :: 作用域限定符　?: 条件运算符
  - . 访问对象成员　* 通过指针间接访问变量
  - # 编译器预处理命令的开始符号
- 不是运算符的符号不能重载，如“;”

2. 运算符重载

运算符重载的概念

```
class C{
  float re;
  float im;
public:
  C operator+(C& c){…}
};
```

```
C c1,c2,c3;
c3=c1+c2;
```

- 目的：使程序更简洁，表达式更直观，增强可读性
- 运算符重载的实质是函数重载：
  - 运算符是函数
  - 运算表达式中的运算符转化为对运算符函数的调用
  - 表达式中的运算对象转化为运算符函数的实参
- 运算符函数名：operator运算符

返回类型 operator 运算符号（参数表）
{ 重载定义 }

2. 运算符重载

运算符重载的实现方式

例：复数的加操作

```
class Complex
 { double re, im;
   public:
     Complex(double r=0.0, double i=0.0)
       { re=r;  im=i;  }
    Complex add(Complex& c)
      {Complex t;
      t.re = re + c.re;
      t.im = im + c.im;
      return t;   }
};

void main( )
{  Complex c1(1, 2), c2(3, 4);
   Complex c3 = c1.add(c2); }
```

- c3 = c1.add(c2);这种使用方式不太直观。
- 我们更希望是如下方式：

  c3=c1+c2;

如何实现？？

2. 运算符重载

运算符重载的实现方式

两种重载方式：

1、重载为类的成员函数

当运算符函数属于某个特定的类时，将此重载函数声明为类的成员函数。【有this指针，第一个参数为this指针所指对象，第二个参数(如果有)通过函数参数传递】

2、重载为友元函数

当运算符函数不属于某个类时，一般将此重载函数声明为类的友元函数。【无this指针，所有参数通过函数参数传递】

2. 运算符重载

重载为类的成员函数

```
class Complex
 { double re, im;
   public:
     Complex(double r=0.0,
       double i=0.0)
       { re=r; im=i;}
    Complex add(Complex& c)
      {Complex t;
        t.re = re + c.re;
        t.im = im + c.im;
        return t;   }
};
void main( )
{  Complex c1(1, 2), c2(3, 4);
   Complex c3 = c1.add(c2); }
```

```
class Complex
 { double re, im;
   public:
     Complex(double r=0.0, double
       i=0.0)
       { re=r; im=i;}
    Complex operator+(Complex& c)
      {Complex t;
        t.re = re + c.re;
        t.im = im + c.im;
       return t;   }
};
void main( )
{ Complex c1(1, 2), c2(3, 4);
  Complex c3 = c1 + c2;  //相当于
      c3 = c1.operator+(c2) }
```

重载为类的成员函数

2. 运算符重载

重载为友元函数

```
class Complex
 { double re, im;
   public:
     Complex(double r=0.0, double
       i=0.0)
       { re=r; im=i;}
   Complex operator+(Complex& c)
      {Complex t;
      t.re = re + c.re;
      t.im = im + c.im;
     return t;   } };
```

```
void main( )
{ Complex c1(1, 2), c2(3, 4);
  Complex c3 = c1 + 27;  //OK
  Complex c3 = 27+ c1;  //ERROR
 }
```

重载为类的成员函数

◈ 用成员函数重载运算符会碰到麻烦：

c3 = c1 + 27相当于: c3=c1.operator+(Complex(27))

c3 =27+c1被解释为无意义: c3=27.operator+(c1)

2. 运算符重载

重载为友元函数

```
#include<iostream.h>
class Complex
{ public:double re, im;
   public:
     Complex(double r=0.0, double i=0.0)
       { re=r; im=i;}
  friend Complex operator+(Complex&, Complex&); //友元函数声明
  };
Complex operator+(Complex& c1, Complex& c2)  //友元函数定义
      {Complex t;
      t.re = c1.re + c2.re;
      t.im = c1.im + c2.im;
      return t;   }
void main( )
{ Complex c1(1, 2), c2(3, 4);
  Complex c3 = c1 + 27;  //OK, 相当于c3=operator+(c1,Complex(27))
  Complex c3 = 27+ c1;  //OK, 相当于c3=operator+(Complex(27), c1)
 }
```

重载为友元函数

2. 运算符重载

两种运算符重载函数对比

```
friend Complex operator+(Complex& c1, Complex& c2) //重载为友元函数
    { Complex t;
      t.re = c1.re + c2.re;
      t.im = c1.im + c2.im;
      return t;  }

Complex operator+(Complex& c) //重载为类的成员函数
    {  Complex t;
      t.re = re + c.re;   //相当于t.re=this->re+c.re;
      t.im = im + c.im; //相当于t.im=this->im+c.im;
      return t;   }
```

用友元或成员函数重载运算符的一般原则

- 没有强制要求必须用哪种种方式重载运算符
- 如果运算符的操作需修改类对象的状态，应实现为成员函数
- 如果运算符所需操作数(尤其是第一个操作数)希望有隐式类型转换，则应实现为友元函数
- 必须重载为成员函数：=、 () 、[]、->
- 运算符重载应根据语义来确定运算符重载函数的返回值。合适的返回类型将会使得：
  - 重载的“=”运算符可以级联：a=b=c。
  - 能够进行诸如a+b+c+d这样复杂的运算。
  - 能够进行cout<<a<<b或cin>>a>>b这样的级联。

2. 运算符重载

例：重载=、＋运算符

```
#include<iostream.h>
class counter
{ int value;
  public:
    counter& operator=(int num);//表示 对象＝整数
    friend counter operator+(counter obj,int num); //表示 对象+整数
    friend counter operator+(int num,counter obj); //表示 整数+对象
};

counter& counter::operator=(int num)
{ value=num;  return *this; }

counter operator+(counter obj,int num)
{ counter tmp;  tmp.value=obj.value+num;   return tmp; }

counter operator +(int num,counter obj)
{ counter tmp;  tmp.value=obj.value+num;  return tmp; }

void main( )
{   counter obj1;
    obj1=10;//使用重载的＝运算符
    counter obj2=obj1+5;//使用重载运算符函数operator+(counter,int)
    counter obj3=3+obj2; //使用重载运算符函数operator+(int, counter)
}
```

2. 运算符重载

例：重载++运算符

```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    Point operator++( )//用成员函数重载
    {  x++;  y++;
       return *this; }

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
      return 0; }
};

void main( )
{
    Point point1(1,3);
    Point point2=++point1;
    point1.output(); point2.output();
}
```

```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    //用友元函数和重载
    friend Point operator++( Point obj);

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
     return 0;  }
};

Point operator++(Point obj)
{ obj.x++; obj.y++;
  return obj; }

void main( )
{
    Point point1(1,3);
    Point point2=++point1;
    point1.output(); point2.output();
}
```

传对象

传对象

2. 运算符重载

例：重载++运算符

```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    Point operator++( )//用成员函数重载
    {  x++;  y++;
       return *this; }

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
      return 0; }
};

void main( )
{
    Point point1(1,3);
    Point point2=++point1;
    point1.output(); point2.output();
}
```

```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    //用友元函数和重载
    friend Point operator++( Point obj);

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
     return 0;  }
};

Point operator++(Point obj)
{ obj.x++; obj.y++;
  return obj; }

void main( )
{
    Point point1(1,3);
    Point point2=++point1;
    point1.output(); point2.output();
}
```

传对象

传对象

2. 运算符重载

例：重载++运算符（二）

```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    //用友元函数和重载
    friend Point operator++( Point obj);

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
     return 0;  }
};

Point operator++(Point obj)
{ obj.x++; obj.y++;
  return obj; }

void main( )
{
    Point point1(1,3);
    Point point2=++point1;
    point1.output(); point2.output();
}
```

```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    //用友元函数和重载
    friend Point operator++( Point &obj);

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
     return 0;  }
};

Point operator++(Point &obj)
{ obj.x++; obj.y++;
  return obj; }

void main( )
{
    Point point1(1,3);
    Point point2=++point1;
    point1.output(); point2.output();
}
```

传引用

传引用

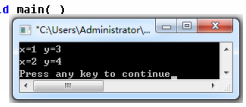
2. 运算符重载

例：重载++运算符（二）

```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    //用友元函数和重载
    friend Point operator++( Point obj);

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
     return 0;  }
};

Point operator++(Point obj)
{ obj.x++; obj.y++;
  return obj; }

void main( )
{
}
```

传对象

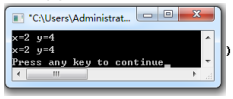
传对象




```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    //用友元函数和重载
    friend Point operator++( Point &obj);

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
     return 0;  }
};

Point operator++(Point &obj)
{ obj.x++; obj.y++;
  return obj; }

void main( )
{
}
```

传引用

传引用



2. 运算符重载

一元和二元运算符的重载

一元运算符：@

作为成员函数重载时的定义及调用方式：

type ClassName::operator@();
ClassName obj;
@obj 或 obj@ ；

运算符重载函数operator@的参数表为空，因为所需的一个操作数通过this指针隐含传递

作为友元函数重载时的定义及调用方式为：

type operator@(one_parameter
ClassName obj;
@obj 或 obj@ ；

此时运算符重载函数operator@所需的一个操作数在参数表中

2. 运算符重载

一元和二元运算符的重载

二元运算符：@

作为成员函数重载时的定义及调用方式为：

type ClassName::operator@(param);
ClassName obj1,obj2;
obj1@obj2;

运算符重载函数operator@所需的第一个操作数通过this指针隐含传递，第二个操作数通过参数提供。

作为友元函数重载时的定义及调用方式为：

type operator@(param1,param2);
ClassName obj1,obj2;
obj1 @ obj2;

运算符重载函数operator@所需的两个操作数均通过参数提供。

2. 运算符重载

重载++/--运算符

- 运算符++和--有两种使用方式：前缀和后缀。因此在重载这两个运算符时，应该考虑到这两种方式的差别
- 二者的差别主要在于函数的参数不同
- 为了方便起见，我们仅以++为例。

前缀: ++obj

成员函数方式

type ClassName::operator++();

友元函数方式

type operator++(ClassName&);

后缀: obj++

成员函数方式

type ClassName::operator++(int);

友元函数方式

type operator++(ClassName&, int);

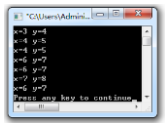
这里的int类型参数叫做占位参数，用来区别前缀和后缀方式，一般在函数体里不会引用它。

2. 运算符重载

重载++/--运算符

```
#include<iostream.h>
class Point
{ int x, y;
  public:
    Point(int a=0,int b=0):x(a),y(b){ }

    friend Point operator++( Point &obj);

    friend Point operator++( Point &obj,int);

    int output()
    {cout<<"x="<<x<<" y="<<y<<endl;
     return 0;  }
};

Point operator++(Point &obj)
{ obj.x++; obj.y++;
  return obj;}

Point operator++(Point &obj , int)
{  Point tmp=obj;//请注意该语句
   obj.x++; obj.y++;
   return tmp; //请注意该语句
}
```

```
void main( )
{ Point point1(3,4);
  point1.output();
  (++point1).output();
  (point1++).output();

  Point point3=++point1;
  point1.output();
  point3.output();

  Point point2=point1++;
  point1.output();
  point2.output();
}
```



2. 运算符重载

重载下标运算符[]

[ ]——双目运算符

X[Y]:

X——左操作数

Y——右操作数

✓运算符重载函数：

operator[ ]

✓设X是某个类的对象，X[Y]相当于：

X.operator[ ](Y)

✓重载[ ]不可以用友元函数。

```
#include<iostream.h>
class IntArray4
{ int A[4];
  public:
    IntArray4(int a1,int a2,int a3,int a4)
    {A[0]=a1;A[1]=a2;A[2]=a3;A[3]=a4; }
};

void main( )
{ IntArray4 A(1,2,3,4);
  cout<<A[2];//需要通过类的对象访问
  cout<<A.A[2]<<endl;}//不可访问类的私有成员
```

```
#include<iostream.h>
class IntArray4
{ public:int A[4];//改为公有成员
  public:
    IntArray4(int a1,int a2,int a3,int a4)
    {A[0]=a1;A[1]=a2;A[2]=a3;A[3]=a4; }
};

void main( )
{ IntArray4 A(1,2,3,4);
  cout<<A.A[2]<<endl;} // OK
```

2. 运算符重载

重载下标运算符[]

[ ]——双目运算符

X[Y]:

X——左操作数

Y——右操作数

✓运算符重载函数：

operator[ ]

✓设X是某个类的对象，X[Y]相当于：

X.operator[ ](Y)

✓重载[ ]不可以用友元函数。

```
#include<iostream.h>
class IntArray4
{ int A[4];
  public:
    IntArray4(int a1,int a2,int a3,int a4)
    {A[0]=a1;A[1]=a2;A[2]=a3;A[3]=a4; }

    //重载[]
    int &operator []()int bi)
    {if (bi<0||bi>=4)
        cout<<"数组下标越界!"<<endl;
        return A[bi];}
};

void main( )
{ IntArray4 A(1,2,3,4);
  cout<<A[2]<<endl;//直接用对象名加下标访问
  A[3]=8;//如果重载函数返回类型为int,会怎样?
  cout<<A[3]<<endl;
 }
```

2. 运算符重载

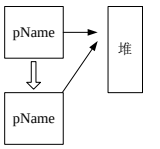
重载赋值运算符=

```
void main( )
{  Complex c1(1, 2), c3;
   Complex c4=c1; //调用的是拷贝构造函数
   c3= c1;} //调用的是重载的赋值运算符
```

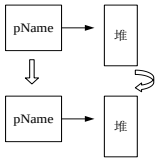
◈赋值运算符与拷贝构造函数的异同

相同点：它们的目的都是为了将一个对象的数据成员复制到另一个对象中。

不同点：拷贝构造函数是要创建一个新对象，而赋值运算符则是要改变一个已经存在的对象的值。



位拷贝(浅拷贝)



值拷贝(深拷贝)

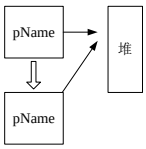
2. 运算符重载

重载赋值运算符=

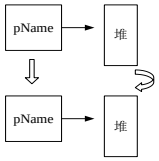
✓如果用户没有在类中重载赋值运算符，那么编译器会生成一个缺省的赋值运算符，该缺省的赋值运算符的作用就是通过位拷贝的方式将源对象复制到目的对象。

✓当用户自定义类要重载赋值运算符时，必须重载为成员函数：

```
X& X::operator=(X & from)
  {
    //复制X的成员
  }
```



位拷贝(浅拷贝)



值拷贝(深拷贝)

举例（一）

```
class Complex
{ double re, im;
  public:
     Complex(double r=0.0, double i=0.0): re(r), im(i){ } //构造函数
     Complex(Complex &obj) //拷贝构造函数
      { re=obj.re;  im=obj.im;}
     Complex & operator=(Complex &obj) //重载赋值运算符
      { re=obj.re;  im=obj.im;  return *this; }
     ……
};
void main( )
{      Complex c1(1, 2), c2(3, 4),c3;
       Complex c4=c1; //调用的是拷贝构造函数
       c3= c2; //调用的是重载的赋值运算符
}
```

拷贝构造函数和重载赋值运算符函数都可以省略，因为这里不涉及资源的拷贝

举例（二）

```
#include<iostream.h>
#include<string.h>
class A
{ int len; char *buf;
  public:
    A(int n)
    { len=n;  buf=new char[n];}

    A(const A&obj)
    { len = obj.len;
      buf=new char[ len ];
      strcpy( buf , obj.buf );}

    ~A( )
    { delete [ ]buf;}

    void copy(char *src)
    { strcpy(buf,src); }
};
```

```
void main()
{ A  obj1( 64 ), obj2(32);
  obj1.copy("helloworld");
  A obj3=obj1;
  obj2=obj3;//将会出错
}
```

obj2中buf成员指向的32字节的内存区将会丢失

举例（二）_改进

```
#include<iostream.h>
#include<string.h>
class A
{ int len; char *buf;
  public:
    A(int n)
    { len=n;  buf=new char[n];}

    A(const A&obj)
    { len = obj.len;
      buf=new char[ len ];
      strcpy( buf , obj.buf );}

    ~A( )
    { delete [ ]buf;}

    A&operator=(const A&from);

    void copy(char *src)
    { strcpy(buf,src); }
};
```

```
A&A::operator=(const A&from)
{ //避免自己给自己赋值
  if(this==&from) return *this;

  delete [ ]buf;
  len = from.len;
  buf=new char[ len ];
  strcpy( buf , from.buf );
  return *this;
}

void main()
{ A  obj1( 64 ), obj2(32);
  obj1.copy("helloworld");
  A obj3=obj1;
  obj2=obj3; // OK
}
```

增加赋值运算符重载函数

2. 运算符重载

类型转换

- C++允许的类型转换有四种：
  - 标准类型-〉标准类型 → 显式转换 / 隐式转换

```
int x=5;
float y=1.0f;
x=y;
x=int(y);
```

  - 标准类型-〉类类型 → 定义构造函数 / 重载赋值运算符
  - 类类型-〉标准类型
  - 类类型-〉类类型

  → 类型转换函数

2. 运算符重载

类型转换

标准类型转换到类类型

方法：(1)重载“=“运算符；(2)定义构造函数。

```
class counter
{ int value;
  public:
    counter(int v=0)//构造函数
    { value=v; }
    counter&operator=(int num)//重载赋值运算符
    { value=num;  return *this; }
};

void main( )
{   counter obj1=6;//使用构造函数
    counter obj2; //使用构造函数
    counter obj3(6);//使用构造函数
    obj2=10;//使用重载的＝运算符
}
```

2. 运算符重载

类型转换

- C++允许的类型转换有四种：
  - 标准类型-〉标准类型 → 显式转换 / 隐式转换
  - 标准类型-〉类类型 → 定义构造函数 / 重载赋值运算符
  - 类类型-〉标准类型
  - 类类型-〉类类型

  → 类型转换函数

2. 运算符重载

类型转换

类类型转换为其他类型

方法：类型转换函数

```
class counter
{ int value;
  public:
    //构造函数
    counter(int v=0)
    { value=v; }

    //重载赋值运算符
    counter&operator=(int num)
    { value=num;  return *this; }

    //类型转换函数
    operator int( )
    { return value; }
};
```

```
void main( )
{   counter obj1=6;
    counter obj2;
    counter obj3(6);
    obj2=10;

//以下均使用类型转换函数
    int v1=obj1;
    int v2=(int)obj2;
    int v3=int(obj2);
}
```

2. 运算符重载

类型转换

类类型转换为其他类型

方法：类型转换函数

```
className::operator type()
{
        ...
        return type类型的实例;
}
```

- 功能：将className类型的对象转换为type类型的实例。type可以是基本类型，也可以是用户自定义类型。
- 参数与返回值：转换函数无参数，无返回类型，必须有返回type类型实例的return语句
- 归属：只能定义为类的成员函数

第五章 多态性

1. 基本概念
2. 运算符重载
3. 虚函数
4. 抽象类

3. 虚函数

虚函数与运行时的多态性

```
class B{
public:
  void f1(){…}
};
```

```
class D: public B{
public:
  void f2(){…}
};
```

- 基类对象的指针可以指向派生类对象

```
D d;
B* pb=&d;
```

- 利用p可以访问从基类B继承的成员，但不能访问派生类自己定义的新成员

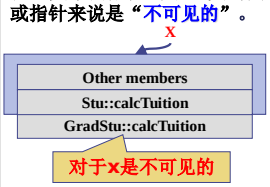
```
pb->f1();   //正确
pb->f2();   //错误
```

3. 虚函数

虚函数与运行时的多态性

```
class Stu
{public:
   float calcTuition()
      {…}    };

class GradStu:public Stu
{public:
   float calcTuition()
      {…}    };

void fn(Stu & x)
  {x.calcTuition();}

int main()
{ Stu s; GradStu gs;
  fn(s);  //计算学生s的学费
  fn(gs); //计算研究生gs的学费???
}
```

◈目的是通过基类引用或指针访问派生类的成员。但行不通的，因为通过基类引用或指针所能看到的是一个基类对象，派生类中的成员对于基类引用或指针来说是“不可见的”。



父类指针和引用可指向子类对象;但仍调用Stu类的calcTtuition

3. 虚函数

虚函数与运行时的多态性

```
class Stu
{public:
   float calcTuition()
      {…}    };

class GradStu:public Stu
{public:
   float calcTuition()
      {…}    };

void fn(Stu & x)
  {x.calcTuition();}

int main()
{ Stu s; GradStu gs;
  fn(s);  //计算学生s的学费
  fn(gs); //计算研究生gs的学费 ???
}
```

✓如何让fn(s)调用Stu中函数，fn(gs)调用GradStu中函数？？？

✓虚函数…

3. 虚函数

虚函数的定义

```
class Stu
{public:
   virtual float calcTuition()
      {…}    };

class GradStu:public Stu
{public:
   float calcTuition()
      {…}    };

void fn(Stu & x)
  {x.calcTuition();}

int main()
{ Stu s; GradStu gs;
  fn(s);  //计算学生s的学费
  fn(gs); //计算研究生gs的学费
}
```

✓虚函数是成员函数，用关键字virtual 来说明函数是虚函数，说明虚函数的方法：

virtual 类型说明符 成员函数名（参数表）

✓关键字virtual只要在基类中说明某一函数，在后面的派生类中被重新说明的函数将自动变成虚函数，而不管有没有使用virtual修饰符【父类中用virtual，子类中可省略】

3. 虚函数

虚函数的定义

```
class Base
{ public:
  virtual void fn()
    {cout <<"In Base class\n"; }
  };
class SubClass :public Base
{ public:
  virtual void fn() // virtual可省略
  { cout <<"In SubClass\n"; }
};
void test(Base& b)
  {  b.fn(); }
```

```
int main()
{
  Base bc;
  SubClass sc;
  cout <<"Calling test(bc)\n";
  test(bc);
  cout <<"Calling test(sc)\n";
  test(sc);
}
```

由于fn()为虚函数，编译器看见test()中的b.fn()后，将其作为滞后联编来处理，以保证在运行时确定调用哪个fn()虚函数。

3. 虚函数

虚函数实例——计算几何图形面积

```
class Shape  //基类
{ public:   Shape(double x, double y) :xCoord(x), yCoord(y) {…}
            virtual double Area() const {return 0.0;}  //虚函数
  protected:  double xCoord, yCoord; };
class Circle :public Shape   //派生类
{ public:   Circle(double x, double y, double r):Shape(x, y), radius(r) {…}
        virtual double Area() const {return 3.14*radius*radius;} //虚函数
protected:  double radius;
};
class Rectangle :public Shape  //派生类
{ public:Rectangle(double x1, double y1, double x2, double y2)
                :Shape(x1,y1), x2Coord(x2), y2Coord(y2){}
        virtual double Area() const; //虚函数
  protected: double x2Coord, y2Coord; };
double Rectangle::Area() const   //派生类Rectangle的Area()函数定义
  {return fabs((xCoord-x2Coord)*(yCoord-y2Coord));}
```

常成员函数：在常成员函数中不能改变数据成员的值

3. 虚函数

虚函数实例——计算几何图形面积

```
void fun(const Shape & sp)
  {cout <<sp.Area() <<endl;}

int main()
 { Circle c(2.0,5.0,4.0);
    Rectangle t(2.0,4.0,1.0,2.0);
    fun(c);
    fun(t);
}
```

```
int main()  //指针版
{ Shape *ps;
  Circle c(2.0,5.0,4.0);
  ps=&c;
  cout<<ps->Area()<<endl;
  Rectangle t(2.0,4.0,1.0,2.0);
  ps=&t;
  cout<<ps->Area()<<endl;
  }
```

将计算面积的函数定义为虚函数，fun()函数可以用来计算所有对象的面积。其它的工作由滞后联编来完成，这样以后要增加一个求新的形状的面积，只要简单地派生一个类就行了，应用程序不用作修改。

3. 虚函数

重要说明

✓实现动态联编需要以下几个条件

1. 虚函数。

2. 通过基类对象的**指针**或者**引用**调用虚函数。如直接采用对象调用虚函数，则采用静态联编方式调用虚函数。

3.虚函数的原型必须一致。

因为只有父类的指针和引用可以指向子类的对象；一般类型的对象所属类是确定的。

```
void fun(const Shape & sp)
  {cout <<sp.Area() <<endl;}
```

✓不恰当的虚函数及例外

1. 不恰当的虚函数:——如果在基类与子类中出现的仅仅是名字的相同，而参数类型不同，或返回类型不同，即使写上了virtual关键字，则也不进行动态联编。

2. 例外——如果基类中的虚函数返回一个**基类指针或引用**，子类中的虚函数返回**一个子类指针或引用**，则C++将其视为同名虚函数进行动态联编。

3. 虚函数

重要说明

✓实现动态联编需要以下几个条件

1. 虚函数。

2. 通过基类对象的**指针**或者**引用**调用虚函数。如直接采用对象调用虚函数，则采用静态联编方式调用虚函数。

3.虚函数的原型必须一致。

virtual Shape &fun() ——基类成员函数
virtual Circle &fun()——派生类成员函数
virtual Rectangle &fun()——派生类成员函数

✓不恰当的虚函数及例外

1. 不恰当的虚函数:——如果在基类与子类中出现的仅仅是名字的相同，而参数类型不同，或返回类型不同，即使写上了virtual关键字，则也不进行动态联编。

2.例外——如果基类中的虚函数返回一个**基类指针或引用**，子类中的虚函数返回**一个子类指针或引用**，则C++将其视为同名虚函数进行动态联编。

3. 虚函数

虚析构函数

```
class A
 { …
   ~A();
   …};

class B:public A
 { …
   ~B();
   …};

A*pObj = new B;
…
delete pObj; //将会调用A类的析构函数
```

✓当使用一个父类的指针指向一个子类对象时，除了会在成员函数的调用上产生问题外，也会在对象的析构上产生问题。

✓由于pObj是A类的指针，使用delete删除pObj所指向的动态对象时，不管pObj指针指向的是A类的对象还是B类的对象，C++语言将会调用A类的析构函数。

3. 虚函数

虚析构函数

```
class A
 { …
   virtual ~A();
   …};

class B:public A
 { …
   virtual ~B();
   …};

A*pObj = new B; …
delete pObj; //将会调用B类的析构函数
```

✓当pObj指向的是B类对象时，就必须将析构函数定义为虚拟函数。这样就能够安全地使用delete来删除对象了，C++语言将调用B类的析构函数。

✓在需要多态性质的任何场合，不管析构函数是不是起作用，习惯上都把析构函数定义成虚拟函数。

✓默认的：Visual C++生成新类时，会自动在析构函数前加上virtual 关键字。

3. 虚函数

虚函数的限制

✓**一个类中将所有的成员函数都尽可能地设置为虚函数总是有益的。但会增加时空上的开销。**

但须注意如下事项：

1. 只有类的成员函数才能说明为虚函数。
2. 静态成员函数不能是虚函数。
3. 内联函数不能是虚函数。
4. 构造函数不能是虚函数。
5. 析构函数可以是虚函数，而且通常声明为虚函数。

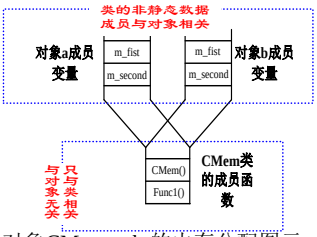
✓C++语言在实现虚拟成员函数的功能时采用了滞后绑定技术和虚拟函数表，这些技术仅涉及编译系统的工作原理和程序运行时的内部实现方法，与实际的应用程序设计关系不大。

3. 虚函数 虚函数实现机制(类及对象内存分配)

✓指针是C语言的灵魂，虚函数是C++语言的灵魂。MFC类库广泛应用了虚函数，增强其通用性。

```
class  CMem
{
  public:
     CMem(){};
     fun1();
     int m_first;
  private:
     char m_second;
};
int main()
{CMem  a, b;}
```

类及对象的内存分配机制

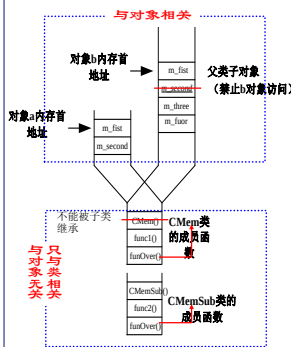


对象CMem a, b;的内存分配图示

3. 虚函数 基类和派生类的内存分配（非虚函数）

```
class  CMem
 {public: CMem(){};
          int fun1();
          int funOver() {return  1;};
          int m_first;
private:   char m_second;
};
class CMemSub : public CMem
  { public: CMemSub(){};
            int fun2();
            int funOver(){return 2;};
            int m_three;
    private: int m_four;
  };
int main()
{CMem  a;  CMemSub b;…}
```

对象a、b的内存分配



3. 虚函数 主函数及结果解释

```
void main()
{
  CMem  a, *pMem;
  CMemSub  b;
  pMem = &a;
  cout<< pMem ->funOver() << endl;
  pMem = &b;
  cout<< pMem->funOver() << endl;
}
```

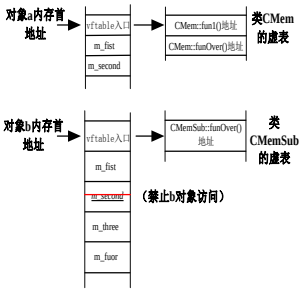
执行后输出：
1
1

✓pMem的类型是Cmem指针，不论pMem指向什么对象的地址，pMem -> funOver始终调用CMem:: pMem -> funOver成员函数

✓对于非虚函数的调用，编译器只根据指针的**数据类型**翻译函数地址，判断调用函数的合法性(即由对象指针的类型来确定，而不由指针所指对象的类型来确定)。因为对象的内存地址空间中只包含数据成员变量，不包含成员函数的信息，所以非虚拟函数的地址翻译过程与其对象的内存地址无关。

3. 虚函数 基类和派生类的内存分配（虚函数）

```
class  CMem
 {public: CMem(){};
          virtual int fun1();
          virtual int funOver()
            {return  1;};
          int m_first;
private:   char m_second;};
class CMemSub : public CMem
  { public: CMemSub(){};
            int fun2();
            virtual int funOver()
              {return 2;};
            int m_three;
    private: int m_four; };
int main()
{CMem  a;  CMemSub b;…}
```



对象CMem a; CMemSub b; 的内存分配

3. 虚函数 主函数及结果解释

```
void main()
{CMem  a, *pMem;
  CMemSub  b;
  pMem = &a;
  cout<< pMem ->funOver() << endl;
  pMem = &b;
  cout<< pMem->funOver() << endl;
}
```

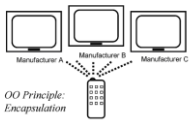
执行后输出：
1
2

语句pMem =&b;使pMem指向对象b的内存空间，调用pMem->funOver时，编译器得到了对象b的vftable入口，并由这个入口找到了相应的虚函数地址。

对于虚函数，它的地址翻译取决于**对象的内存地址**，而不取决数据类型。如果类中定义了虚函数，该类及其派生类在编译时会给这个类一个vfptr指针指向一张虚函数表vftable，在类的对象地址空间中存储一个该虚表的入口，占4个字节，这个入口地址是在构造对象时由编译器写入的。使用了关键字virtual就可告诉系统不要根据指针的类型访问，而应根据指针实际所指向对象的类型来访问。

3. 虚函数 提供虚函数的意义

- 虚函数的特性使得它成为许多C++程序设计的关键，因为基类可以使用虚函数提供一个接口，这使该类的所有公有派生类都具有相同的接口，但派生类可以定义自己的实现版本，而且虚函数调用的解释依赖于它的对象类型。这实现了“一个接口，多种实现/语义”的概念，为软件的充分可重用提供了坚实的基础。
- 虚函数 ⟶纯虚函数 ⟶抽象类

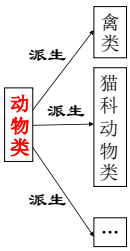


第五章 多态性

1. 基本概念
2. 运算符重载
3. 虚函数
4. 抽象类

4. 抽象类

抽象类的概念



✓抽象类是不能实例化的特殊类。

✓抽象类有一些公共的接口，表示该类对象拥有的共同操作，这些操作依赖于不同的派生类对象，因此，在抽象类的定义中，这些公共接口只需要有说明而不需要有实现。

✓对于抽象类的使用有几点规定：

1. 抽象类只能用作其它类的基类
2. 抽象类不能用作参数类型、函数返回类型或显式转换类型
3. 可以说明指向抽象类的指针和引用，此指针可以指向它的派生类，进而实现多态性

4. 抽象类

纯虚函数

✓包含纯虚函数的类为抽象类。一个抽象类至少包含一个纯虚函数。

✓纯虚函数是一种特殊的虚函数，它的格式：

class 类名

{ virtual 数据类型 成员函数名（参数表）= 0；};

✓在许多情况下，在基类中不能对虚函数给出有意义的实现，而把它说明为纯虚函数，它的实现留给该基类的派生类去做。

✓抽象类虽然不能实例化，但它可以定义整个类家族通用的成员变量和成员函数，为派生类提供一个框架。抽象类为派生类提供了虚拟函数的重载形式，可以用**抽象类的指针引用派生类的对象**，这为虚函数的应用准备了必要的条件。

✓特别地，当抽象类的所有函数成员都是纯虚函数时，这个类被称为**接口类**。

4. 抽象类

抽象类实例

```
class Shape //基类
{ public:   Shape(float x, float y) :m_x(x), m_y(y) {…}
            virtual float Area() const =0;  //纯虚函数
  protected:  float m_x, m_y; };
class Circle :public Shape  //派生类
{ public:   Circle(float x, float y, float r):Shape(x, y), m_r(r) {…}
            float Area() const  //虚函数
            {return 3.14*radius*radius;}
protected:  float m_r;
};
class Rectangle :public Shape  //派生类
{ public:  Rectangle(float x1, float y1, float x2, float y2)
                :Shape(x1,y1), m_x2(x2), m_y2(y2){}
           double Area() const //虚函数
             {return fabs((m_x-m_x2)*(m_y-m_y2));}
protected: float m_x2, m_y2; };
```

常成员函数：在常成员函数中不能改变数据成员的值

4. 抽象类

抽象类实例

```
int main()
{ Shape  s;  // error
  Shape *ps; //不能声明抽象类的对象，但可以声明抽象类的指针

  Circle c(2.0f,5.0f,4.0f);
  ps=&c; //抽象类的指针指向派生类
  cout<<ps->Area()<<endl;

  Rectangle t(2.0f,4.0f,1.0f,2.0f);
  ps=&t;  //抽象类的指针指向派生类
  cout<<ps->Area()<<endl;
 }
```

END